

吴成伟 E2→E3 晋升答辩陈述稿（口述版）

预计时长：约9-10分钟 | 字数：约1800字

建议：打印出来，用口头语多读几遍，形成肌肉记忆

开场（20秒）

各位领导、评委老师好，我是吴成伟，来自SS-Alpha。

今天我来申请晋升E3。接下来，我想从六个方面，跟大家分享一下我这四年多的工作实践和成长思考。

今天的分享我会分六个部分：

第一部分是个人简介，让大家快速了解我的背景；

第二、三部分是重点，我会详细讲CLR项目的架构设计，以及我如何保证高质量交付——这也是我申请E3的核心支撑；

第四部分分享我的技术好奇心和执行力，比如零基础攻关AKS的经历；

第五部分介绍我自己开发的效能工具；

最后是对未来的工作展望。

01 个人简介与目标（1分钟）

先简单介绍一下我自己。

我的技术栈主要是SpringBoot的生态，平时工作用Mybatis、Kafka、Redis比较多。最近两年也在生产环境中落地了Azure AKS容器化部署和CI/CD流程，积累了不少实战经验。

入职以来，我有幸获得了年度优秀员工，帮助公司的软件著作权的申请，也在多个Release里承担过技术攻关的任务。

说到为什么想升E3——我的目标很明确：我希望通过这次晋升，完成从“功能实现者”到“架构设计者”的角色转变。不是停留在完成需求，而是能主动把控技术方案的全局，通过架构升级和质量体系建设，真正提升系统的性能和稳定性，给业务增长创造更大的价值。

02 项目背景与我的架构能力（3分钟）【重点章节】

嗯 好，接下来我想重点讲讲CLR项目，这也是我申请E3的核心支撑。

先快速介绍一下项目背景。（30秒）

CLR是道富公司的数据中转核心系统，负责多Vendor价格数据的统一接入和分发。

简单来说，它是一个“数据枢纽”——上游对接各类Vendor数据源，下游分发给EPW系统和其他子系统，日均处理百万级数据量。

技术栈是SpringBoot + SQLServer + Kafka + Redis。作为数据中转系统，最核心的诉求就两个词：**及时、可靠**。所以我主导设计了失败重试、补偿机制和端到端可监控方案，确保全链路可控。

在CLR项目中的角色定位。（30秒）

作为项目的核心开发，我主要承担以下职责：

1. 参与CLR平台整体的架构设计；
2. 主导数据采集、处理、分发核心模块的开发；
3. 独立完成PriceDiff工具的设计与实现；
4. 推动AKS云平台架构落地。

简单说，我不只是执行开发任务，而是从架构设计到落地部署的全流程主导者。

接下来，重点讲讲我在这个项目里体现的架构能力，以及带来的实际价值。（2分钟）

第一点，业务本质的抽象能力。

面对复杂的数据流转，我做的第一件事是抽象建模。把整个过程提炼成四个核心阶段：**数据接入** → **计算处理** → **存储落地** → **分发路由**。

这个抽象的价值在于：让我能系统性地梳理非功能性需求。

比如性能方面，通过拆解四个阶段，很轻松的定位到分发环节是瓶颈，针对性引入Redis缓存后，接口响应从200ms降到了20ms，直接满足了系统的性能指标要求。

再比如可靠性方面，我识别出数据接入环节的高可用需求，设计了失败重试和补偿机制，确保Vendor数据不丢失。

第二点，技术选型的决策能力。

基于CLR项目的业务特性，我主导了几个关键的技术选型决策，每个都有具体的场景考量：

首先是Kafka的应用。 CLR上游对接多个Vendor，数据来源多样——有些是Kafka推送，有些是FTP文件，还有些是API调用或数据库Query。这些数据流量很不稳定，经常出现突发高峰。如果直接同步处理，下游系统很容易被压垮。我引入Kafka做统一的消息中转层，把各种来源的数据异步缓冲，利用它的削峰填谷能力平滑消费。结果是：即使Vendor端出现流量激增，我们的系统也能稳定处理，下游零故障。

然后是动态配置与Redis缓存的结合。 不同Vendor的数据规则各不相同——换算、映射、校验等。我把这些处理逻辑做成可配置化的，存在SQLServer里，通过Redis缓存实现动态生效。这样不用改代码就能适配新Vendor，实时配置实时上线。

这几个选型决策，都是基于对业务场景的深度理解，而不是盲目追新。

第三点，可扩展性设计。

为了支撑业务的持续增长和可扩展性，我核心模块的开发时进行了：

- **模块化设计：**接入、处理、分发拆成独立模块，某一个模块升级不影响全局，问题排除也能快速定位；
- **配置化驱动：**新Vendor接入只需改配置，零代码改动，缩短开发周期，留给测试人员更长的测试时间；
- **接口标准化：**与下游统一接口契约、解耦下游，新Vendor或新系统接入时，下游无需重复改造，减少工作量。

第四点，性能与可靠性保障。

最后看架构设计的实际收益：

- **Kafka削峰：**缓冲Vendor突发流量，系统零故障运行；
- **Redis缓存：**配置的实时更新与生效；
- **K8s自愈：**主备容错+存活探针，故障Pod自动重启；
- **CI/CD自动化：**打通全流程自动化，提升发布效率。

总结一下我的核心贡献：我参与了CLR平台的整体架构设计，主导了关键模块的接口设计、打通了CI/CD全流程，保障日均百万级数据量的稳定流转，为业务增长和系统稳定提供了保障。

03 我是如何保证高质量交付的（2分钟）【重点章节】

好，架构讲完了，我想再重点分享一下——我在项目里是怎么保证交付质量的。

我主要参与并落地了这一套从需求到交付的质量保障流程。

首先是需求阶段。

- **前置评审：**需求文档一出来，我会主动牵头，拉产品、测试做三方评审，提前对齐认知。这样做的好处是能避免后面的返工。
- **时序建模：**针对核心接口，我会画时序图，把正常流程和异常分支都列清楚，让边界条件一目了然。
- **可行性评估：**我从技术可行性和部署可行性两个维度去评估每个需求。

然后是开发阶段。

- 模块前期由我主导，我制定并推行了统一的编码规范；（逻辑分层、统一异常处理、注释与文档、统一工具类）
- 引入了SonarQube做代码质量扫描，低质量的代码被发送风险报告；
- 对复杂的SQL语句，我会做**性能预审**，避免慢查询上线。

测试阶段也没放松。

- **单元测试：**核心业务代码覆盖率做到了80%以上，用JUnit + Mockito + AssertJ来保证测试质量；
- **集成测试：**编写了业务接口测试用例，生成标准化的测试文档。

部署阶段。

我搭建了Azure DevOps Pipeline，打通了从编译、扫描到部署、AKS的全流程。

最后就是故障演练。

我主动模拟了Pod崩溃、节点宕机、SQL Server连接中断等各种场景，验证系统的自愈能力和风险预警机制。这样做是为了确保——万一生产环境真的遇到这些问题，系统是能扛住的。

总结：我的理解是，高质量的提交物不仅是测出来的，也是“设计”和“管”出来的。

需求理解的零偏差，从源头保证了产出与需求吻合，“故障测试”和“80%测试覆盖率”，将质量前移，确保了我提交的每一个版本都是经过严苛验证的“精品”，而非简单的“功能堆砌”。

04 技术好奇心与执行力（1分钟）

接下来，我想聊聊我的技术好奇心和执行力。

先说AKS的零基础部署。

我个人之前其实没有AKS的使用经验。

- 我自学Kubernetes和Azure的官方文档，完成了AKS集群的搭建；
- 配置了Azure Pipeline实现自动化部署；
- 还设计了ConfigMap + KeyVault的密钥管理方案，解决了配置管理和安全的问题。

然后是系统的高可用加固。

- 我配置了多可用区部署和Pod分散策略，避免单点故障；

- 配置了存活探针（Liveness Probe）和就绪探针（Readiness Probe），实现故障Pod的自动重启。

还有全链路可观测体系。

- 我搭建了结构化日志体系，统一了日志格式和输出规范；
- 设计并实现了Traceld**全链路追踪**，让每个数据请求从接入到分发，全程可查。这个带来的实际效果是——减少定位问题时间。

最后是故障演练验证。

我主动设计并执行了多场景的故障演练：Pod故障、节点下线、数据库连接中断、CPU飙升等等。用来验证高可用设计和监控告警的有效性。

这几点，对应E3标准里的：「具备技术好奇心」「掌握流行和标准工具的备选方案并且能够加以应用」，以及「具有较强的技术执行力」。

总结：回顾这个实践——从零学AKS、把容器化落到生产、主动搞故障演练验证稳定性，我觉得很好地体现了我的技术好奇心与执行力。

05 工具创新与效能提升（1分钟）

在项目开发过程中，我还自己开发一个叫**PriceDiff**的工具，解决业务测试中的数据对比痛点。

具体来说：

- **基于业务的Diff算法：**解决了价格精度差异和格式差异的问题，减少了很多误报；
- **业务主键对齐：**做了定制化的过滤规则和精度适配，更贴合实际业务场景；
- **智能报告生成：**能自动生成带差异标签的Excel报告，变更点一目了然；
- **流水线集成：**接入Azure Pipeline，发布前自动执行校验，如果发现差异会自动发送风险邮件。

这个工具现在已经在团队内部推广使用了，测试效率有明显的提升。

这一点，对应E3的加分项：「能研发提高项目开发效率的小工具」。

06 未来工作展望（1分钟）

如果这次能获得晋升，我想在以下三个方面持续发力：

第一，技术标准化建设。

我想把CLR项目里的模块化经验，沉淀成一个SpringBoot脚手架，统一日志、异常处理、缓存的标准，然后输出到子系统MDW项目中去。

第二，推广架构标准。

我想把CLR的「配置驱动」模式推广到MDW子系统，主导制定组内的微服务设计规范，参与新项目的架构评审。

第三，自动化效能平台。

基于现有的Traceld经验，我想建设一个统一告警中心，实现异常自动定位并推送给负责人，进一步提升运维效率。

结语（30秒）

好，最后我想总结一下。

回顾过去这两年多，我在CLR项目里独立承担了架构设计，建立了一套全流程的质量保障体系，主动攻关了AKS容器化技术，还研发了效能工具。

我觉得这些实践，已经匹配了E3标准对架构能力、技术好奇心、高质量交付和技术执行力的要求。

感谢各位领导和同事的倾听与指导。期待能够获得这次晋升机会，为公司的发展贡献更大的力量。

谢谢大家！

时间分配建议

章节	建议时长	调整说明
开场	20秒	不变
01 个人简介	1分钟	不变
02 架构能力	3分钟	重点展开，背景压缩
03 高质量交付	2分钟	合并重复内容
04 技术好奇心	1分钟	删除重复点
05 工具创新	1分钟	不变
06 未来展望	1分钟	不变
结语	30秒	不变
总计	约9分钟	留1分钟缓冲

附录：E3标准对照表（答辩准备参考）

E3标准要求	本次答辩体现点
技术核心参与多个项目/Release技术攻关或架构设计	CLR平台架构设计、多个Release技术攻关
精通某一技术领域或具有较强架构能力	系统架构章节重点阐述
具备技术好奇心，掌握工具并应用	AKS零基础自学落地、Traceld体系搭建
提交物质量高	全流程质量保障体系
帮助梳理项目需求，保证产出与需求吻合	需求前置评审、时序建模、可行性评估
具有较强的技术执行力	AKS部署、故障演练、CI/CD搭建
熟悉相关系统业务知识（加分项）	业务本质抽象、PriceDiff业务理解
能研发提高项目效率的小工具（加分项）	PriceDiff工具

评委可能的问题

ConfigMap应用场景

案例1：动态日志级别控制`

在生产环境排查问题时，经常需要临时调高日志级别查看详细日志，但又不想重启Pod影响业务。我用ConfigMap管理日志级别配置，配合SpringBoot的@RefreshScope实现热更新。比如某天Vendor数据异常，我直接在ConfigMap把日志级别从INFO改成DEBUG，30秒内生效，排查完再改回去，整个过程服务零中断。

案例2：动态告警阈值调整`

CLR作为核心数据枢纽，监控告警的敏感度需要根据实际情况灵活调整——比如大促期间数据延迟阈值可以适当放宽，而日常则需要严格监控。我把告警阈值配置放到ConfigMap里，配合应用的热加载机制。有次Vendor侧网络抖动导致数据堆积，我直接在ConfigMap调高延迟告警阈值避免误报，问题恢复后再调回原值，全程不用重启服务，告警策略随业务场景动态适配。

案例3：功能开关与灰度发布`

新功能上线时，需要控制风险逐步放量。我把功能开关配置放到ConfigMap里，比如某个新数据源接入功能，先只对特定Vendor开放验证。通过修改ConfigMap里的白名单配置，我可以在不重启服务的情况下，逐步扩大功能覆盖范围。有次新功能在测试环境没问题，但生产环境某个Vendor的数据格式有差异，我立即通过ConfigMap关闭该Vendor的开关，隔离问题后再修复，避免了故障扩散。

否决过2个存在性能隐患的需求方案，当时产品经理还不太理解，但最后证明这个判断是对的。

例子1：技术可行性：跨库JOIN查询实时价格(依赖风险)

有个需求要在价格查询时，实时关联另一个系统的用户权限表做过滤。我评估后发现，这个跨库JOIN查询在高并发下性能极差，而且一旦对方系统网络抖动，我们的查询就会超时。我否决了这个方案，建议用数据同步的方式把权限数据缓存到本地，每天增量同步一次，查询时走本地索引。最终方案既满足了权限控制，又保证了查询性能。

例子2：技术可行性：消息重发次数不设上限(资源耗尽风险)

早期设计失败重试机制时，需求方要求"失败就一直重试直到成功"，理由是数据不能丢。我指出这会导致消息堆积，如果下游系统长时间故障，重试队列会把内存打满，最终拖垮整个服务。我坚持加了最大重试次数限制，超过次数进死信队列人工处理。并发送预警邮件。

案例3：部署可行性：数据库表结构变更需要重启服务`

早期有一个给大表加字段的需求，开发方案里没考虑存量数据迁移，直接上线会导致服务不可用。我在部署评审时指出这个问题，要求补充零停机迁移方案——先加字段允许为空，后台脚本分批回填，最后改约束。虽然开发工作量增加了，但实现了无缝部署上线，用户零感知。

编码规范通常包括：

命名规范

类名用大驼峰（PriceCalculator），方法/变量用小驼峰（calculatePrice）

常量全大写+下划线（MAX_RETRY_COUNT）

包名统一前缀（com.company.clr.module）

代码格式

缩进用4个空格，不换行省略大括号

单行不超过120字符，方法不超过50行

import分组有序（java.* → 第三方 → 本项目）

注释与文档

类/公共方法必须有Javadoc

复杂逻辑加行内注释说明"为什么"而非"做了什么"

接口变更同步更新注释

结构与约束

分层清晰：Controller→Service→Repository，禁止跨层调用

异常统一用自定义业务异常，禁止直接抛RuntimeException

日志统一用SLF4J，禁止System.out.println

最佳实践

数据库连接、流资源必须try-with-resources关闭

并发场景用线程安全类，避免手动synchronized

单元测试覆盖率要求，核心类必须配套测试

分层密钥管理：非敏感配置用ConfigMap，数据库密码等敏感信息通过公司密钥服务动态获取（应用调用服务传入host/user，返回对应密码），既满足安全审计要求，又避免了硬编码密钥的风险

结构化日志：让同一个请求处理线程中的所有日志，自动携带相同的上下文信息（如TraceId）。

1. 日志格式统一
2. 技术实现：Logback + Logstash encoder 统一输出格式
3. TraceId生成与传递，关键节点埋点